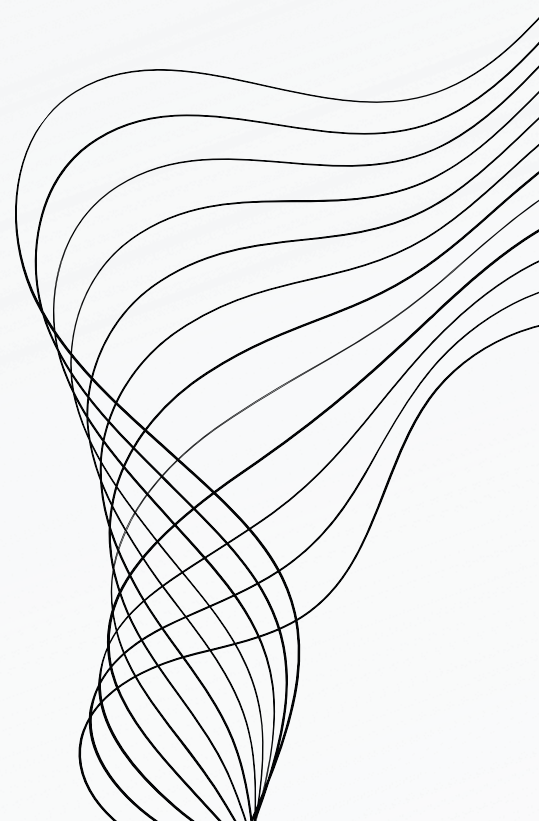




UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

ESTUDO XEOGRÁFICO CAMPUS VIDA

**ADRIÁN QUIROGA LINARES
GUILLERMO FERNÁNDEZ CASTRO**



ÍNDICE

01

RESUMO

02

PROBLEMA

03

FUNCIÓN A ESTUDAR

04

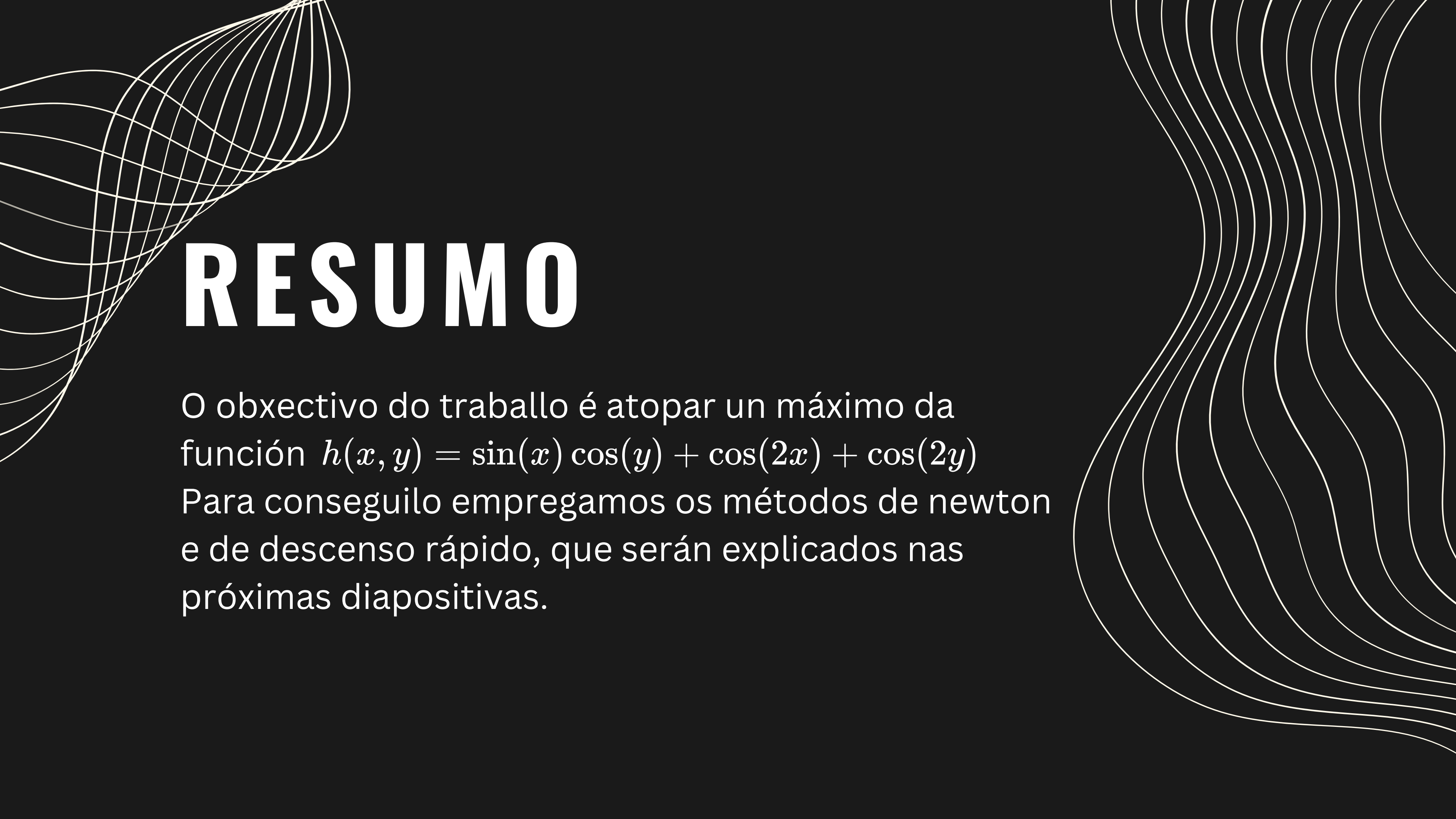
MÉTODO DE NEWTON

05

DESCENSO RÁPIDO

06

WEBGRAFÍA

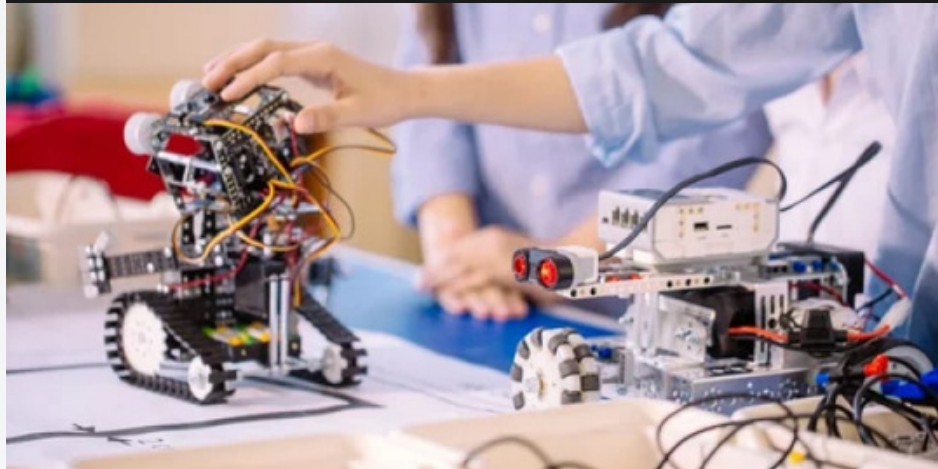


RESUMO

O obxectivo do traballo é atopar un máximo da función $h(x, y) = \sin(x) \cos(y) + \cos(2x) + \cos(2y)$
Para conseguilo empregamos os métodos de newton e de descenso rápido, que serán explicados nas próximas diapositivas.

PROBLEMA

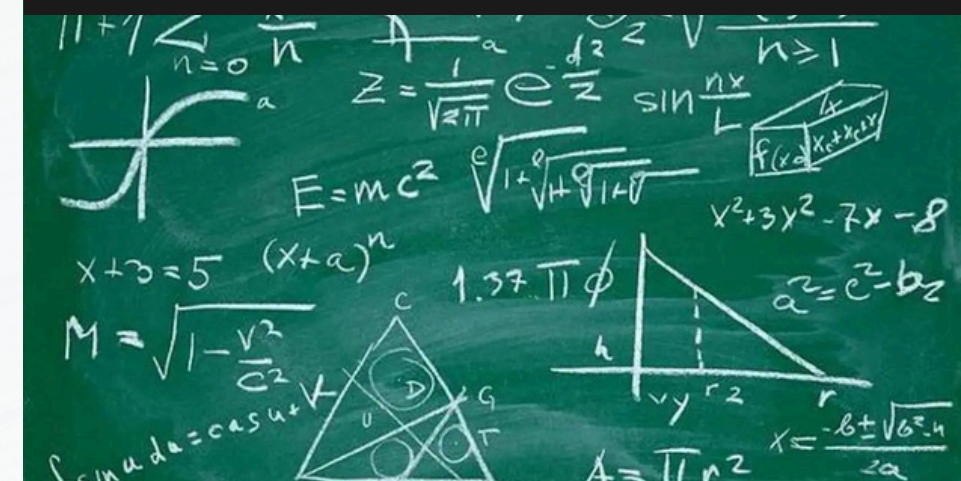
Objetivo



Co paso dos anos e dos avances tecnolóxicos, a Universidade planificou a construción dun novo edificio destinado a impartir clases dun novedoso grao de Robótica no Campus Vida. A topografía do terreo ven dada pola función:

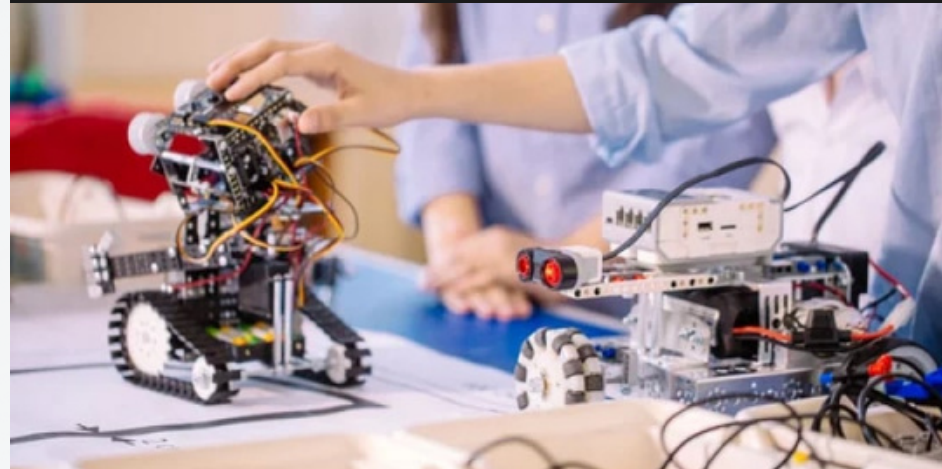
$$h(x, y) = \sin(x) \cos(y) + \cos(2x) + \cos(2y)$$

Fórmula



PROBLEMA

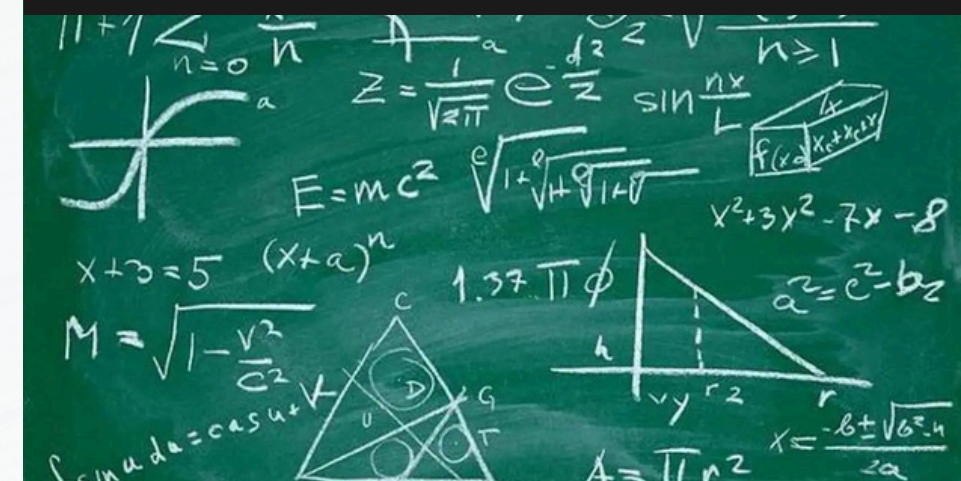
Objetivo



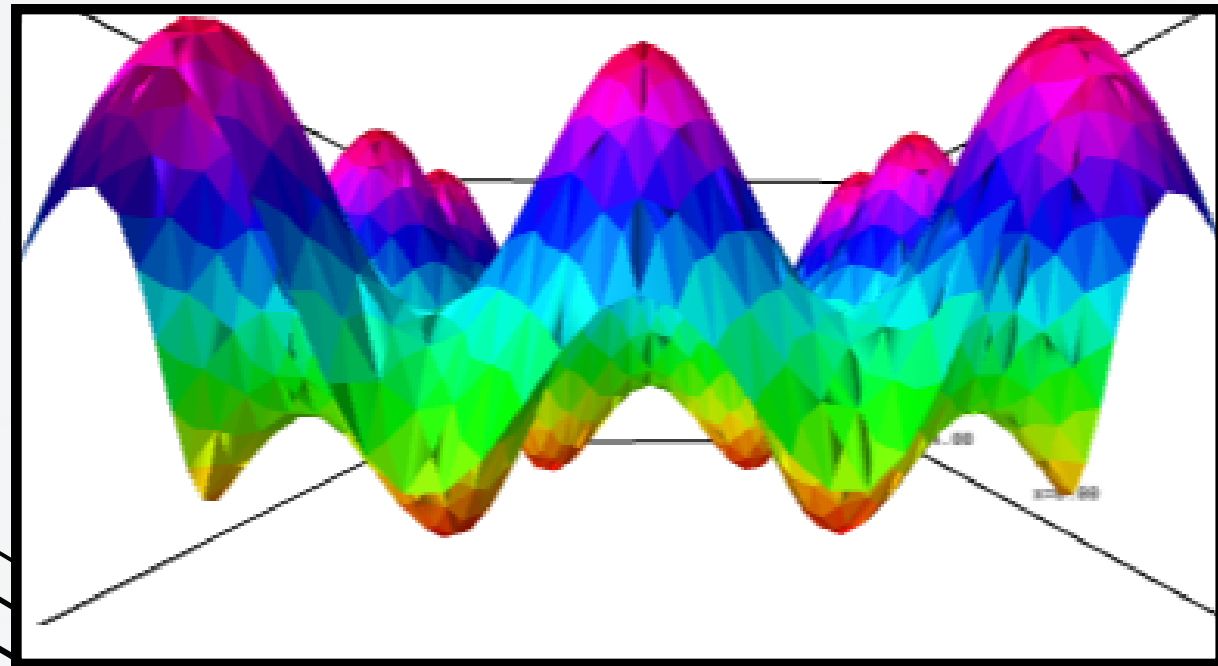
O obxectivo é identificar a ubicación óptima do terreo onde o nivel do chan sexa máximo. Os límites do campus veñen dados por:
[-4,4] para o eixo x
[-4,4] para o eixo y

$$h(x, y) = \sin(x) \cos(y) + \cos(2x) + \cos(2y)$$

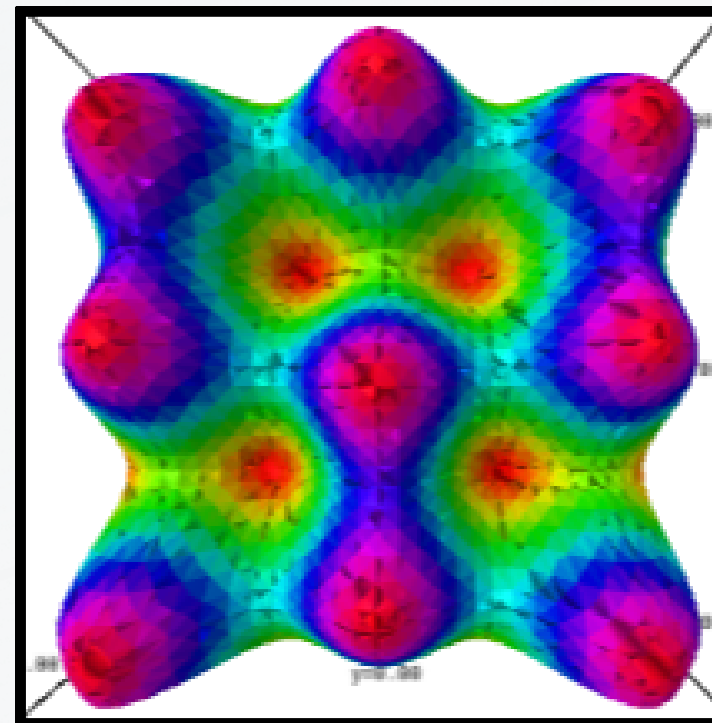
Fórmula



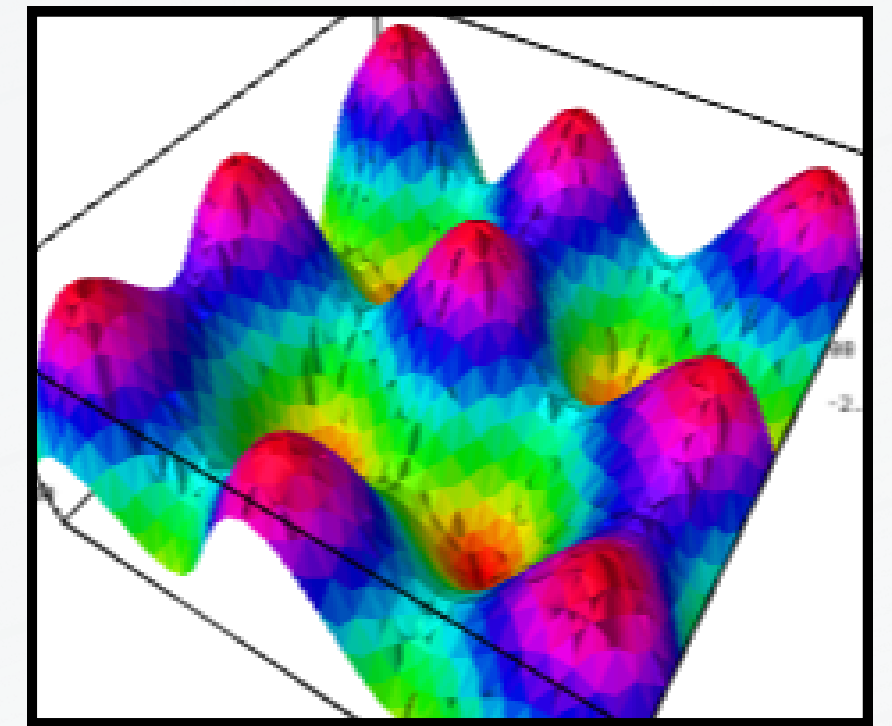
FUNCIÓN A ESTUDIAR



PUNTO DE VISTA
1



PUNTO DE VISTA
2



PUNTO DE VISTA
3

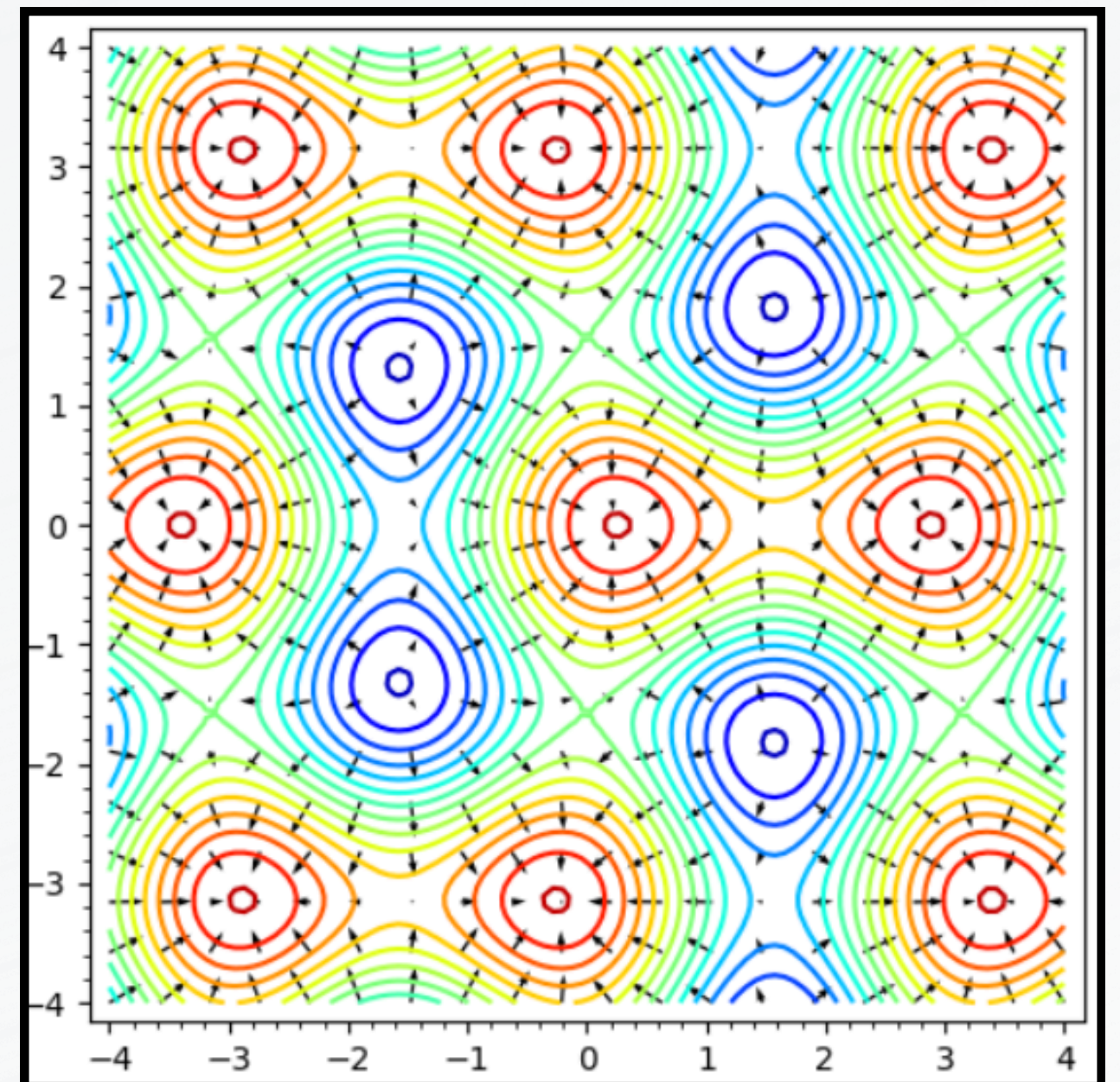
ESTUDO DA FUNÇÃO

- En primeiro lugar calculamos o gradiente da función

$$\nabla h = \left(\frac{\partial h}{\partial x}, \frac{\partial h}{\partial y} \right)$$

```
var('y')  
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)  
args=[(x,-4,4),(y,-4,4)]  
kwds={'fill':False, 'cmap':'jet'}  
contour_plot(h, *args, contours=15, **kwds) + plot_vector_field(diff(h), *args)
```

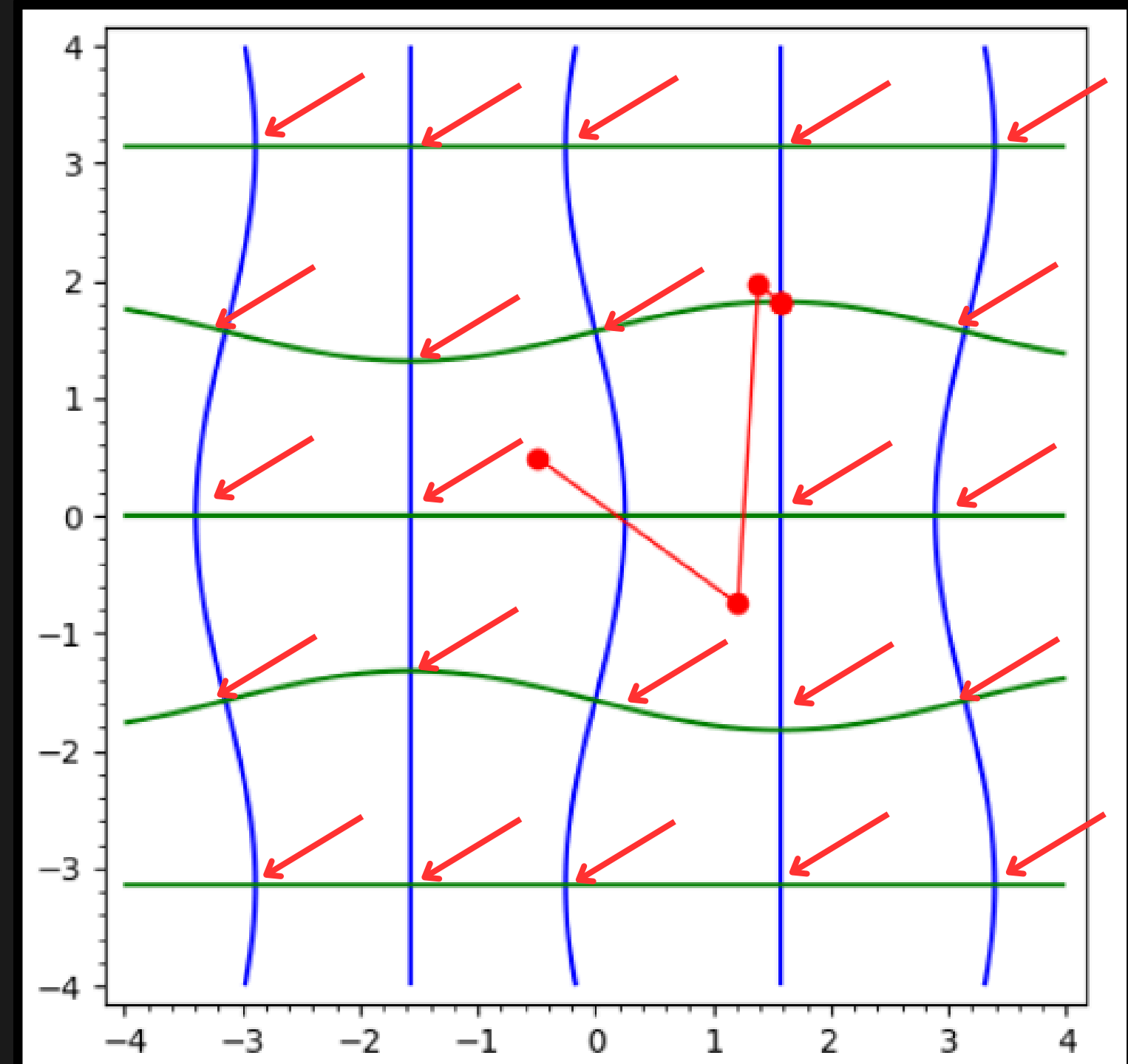
$$\nabla h = (\cos(x) \cos(y) - 2 \sin(2x), -\sin(x) \sin(y) - 2 \sin(2y))$$



CANDIDATOS A MÁXIMOS, MÍNIMOS E PUNTOS DE SELA

Na imaxe podemos observar os conxuntos de nivel 0 das compoñentes do vector gradiente

Esto permítenos observar cales serán os puntos críticos da nosa función




```
def criterio_hessiana(f, x0, xtol=1e-12, rtol=1e-14):  
    """CRITERIO_HESSIANA    Calculo do criterio da hessiana."""  
    if abs(diff(f)(*x0)) > xtol: raise RuntimeError('Non é un punto crítico da función.')  
    H = f.hessian)(*x0)  
    detH = det(H)  
    if abs(detH) < rtol:  
        return 'non decide'  
    elif detH > 0:  
        return 'minimo' if H[0][0]>0 else 'maximo'  
    else:  
        return 'punto de sela'
```

$$H(x, y) = J(x, y)$$

$$H(x, y) = \begin{pmatrix} \frac{\partial^2 h}{\partial x^2} & \frac{\partial^2 h}{\partial x \partial y} \\ \frac{\partial^2 h}{\partial y \partial x} & \frac{\partial^2 h}{\partial y^2} \end{pmatrix} \quad J(x, y) = \begin{pmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial y} \\ \frac{\partial h_2}{\partial x} & \frac{\partial h_2}{\partial y} \end{pmatrix}$$

MÉTODO DE NEWTON

$$J(x, y) = \begin{pmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial y} \\ \frac{\partial h_2}{\partial x} & \frac{\partial h_2}{\partial y} \end{pmatrix} = \begin{pmatrix} -\cos(y) \sin(x) - 4 \cos(2x) & -\cos(x) \cos(y) \\ -\cos(x) \sin(y) & -\cos(y) \sin(x) - 4 \cos(2y) \end{pmatrix}$$

Para poder poñer en práctica o método de Newton é necesario o cálculo da matriz Xacobiana

```
fl = lambda x,y: (cos(x)*cos(y) - 2 * sin(2*x), -sin(x) * sin(y) - 2 * sin(2*y) )  
jacobian(fl(x,y), (x,y))
```

$$\begin{pmatrix} x_{i+1} \\ y_{i+1} \end{pmatrix} = \begin{pmatrix} x_i \\ y_i \end{pmatrix} - (J(x_i, y_i))^{-1} \begin{pmatrix} h_1(x_i, y_i) \\ h_2(x_i, y_i) \end{pmatrix}$$

CÓDIGO DO MÉTODO DE NEWTON

```
def newton(f, df, x0, xtol=1e-12, rtol=1e-14, M=100):
    """NEWTON    Calculo de raices por Newton multidimensional."""
    v = [vector(RDF,x0)]
    for k in range(M):
        dfx = matrix(RDF,df(*v[k]))
        if rank(dfx) < len(x0):
            raise RuntimeError('Matriz xacobiana singular.')
        r = (v[k] - dfx\vector(RDF,f(*v[k])));
        v.append(r)
        # test de parada
        if norm(r - v[k]) < xtol or norm(vector(RDF,f(*r))) < rtol:
            return r, len(v), v
    raise RuntimeError('Excedido numero maximo de iteracions.')
```

```
_,_,v = newton(f, diff(f), (-0.5,0.5))
debuxa_newton(fl, v, args = [(x,-4,4),(y,-4,4)])
```

```
def debuxa_raices(f, args = [(x,-5,5),(y,-5,5)]):
    kwds = {'fill':False, 'contours':[0]}
    return contour_plot(f(x,y)[0], *args, cmap=['blue'], **kwds) + contour_plot(f(x,y)[1], *args, cmap=['green'], **kwds)

def debuxa_newton(f, v, args = [(x,-5,5),(y,-5,5)]):
    return debuxa_raices(f, args) + line(v, marker='o', color='red')
```


MÁXIMO

$\text{Det}(H(x, y)) > 0$ e $\frac{\partial^2 h}{\partial x^2} < 0 \implies$ máximo

$\text{Det}(H(x, y)) < 0 \implies$ punto de sela

$\text{Det}(H(x, y)) > 0$ e $\frac{\partial^2 h}{\partial x^2} > 0 \implies$ mínimo

```
s0 = newton(diff(h), diff(h,2), (-2.2,-3))[0]  
H = h.hessian()(*s0)  
detH = det(H)  
detH
```

15.9375

```
fx(3.394272908731872, -3.141592653589793)
```

-3.7500000000000000

PUNTO DE SELA

$\text{Det}(H(x, y)) > 0$ e $\frac{\partial^2 h}{\partial x^2} < 0 \implies$ máximo

$\text{Det}(H(x, y)) < 0 \implies$ punto de sela

$\text{Det}(H(x, y)) > 0$ e $\frac{\partial^2 h}{\partial x^2} > 0 \implies$ mínimo

```
s0 = newton(diff(h), diff(h,2), (3,2))[0]
```

```
H = h.hessian()(*s0)
```

```
detH = det(H)
```

```
detH
```

```
-17.0
```


MÍNIMO

$\text{Det}(H(x, y)) > 0$ e $\frac{\partial^2 h}{\partial x^2} < 0 \implies$ máximo

$\text{Det}(H(x, y)) < 0 \implies$ punto de sela

$\text{Det}(H(x, y)) > 0$ e $\frac{\partial^2 h}{\partial x^2} > 0 \implies$ mínimo

```
s0 = newton(diff(h), diff(h,2), (-0.5,0.5))[0]  
H = h.hessian()(*s0)  
detH = det(H)  
detH
```

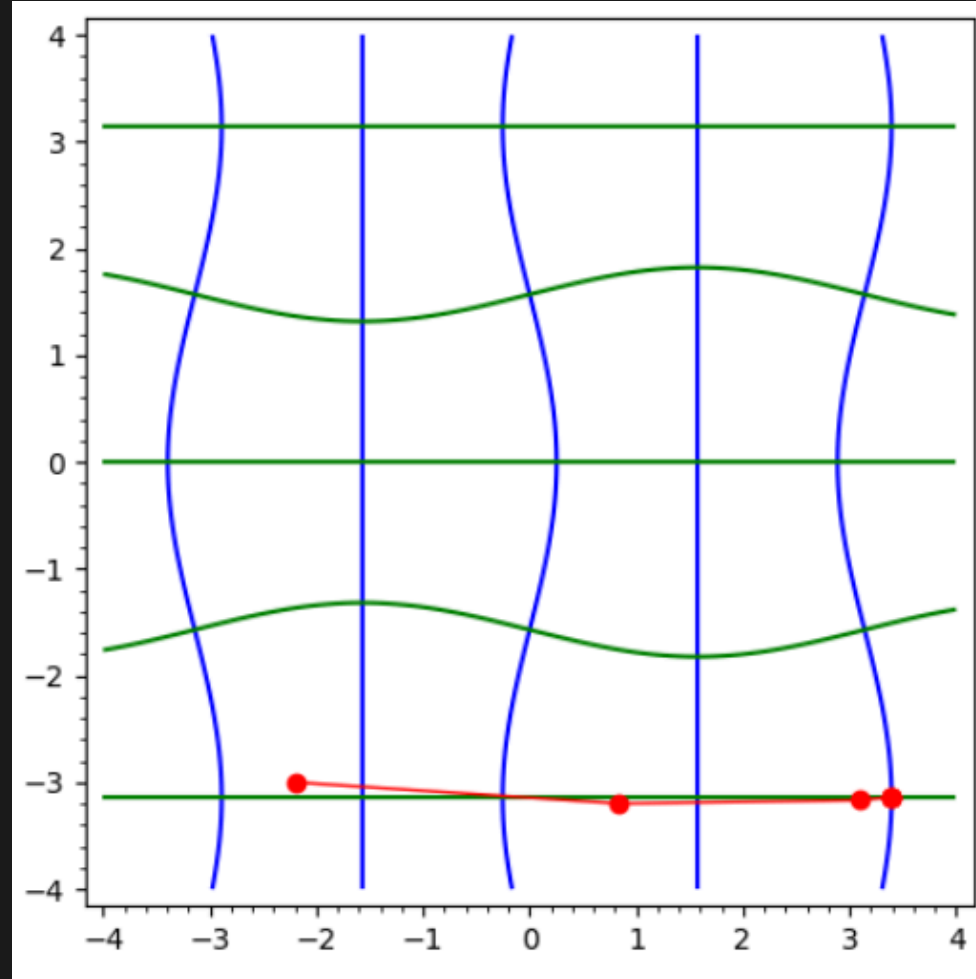
15.937499999999998

```
fxx(x,y)=diff(diff(h,x),x)  
fxx(-1.5707963291809788,1.3181160678957924)
```

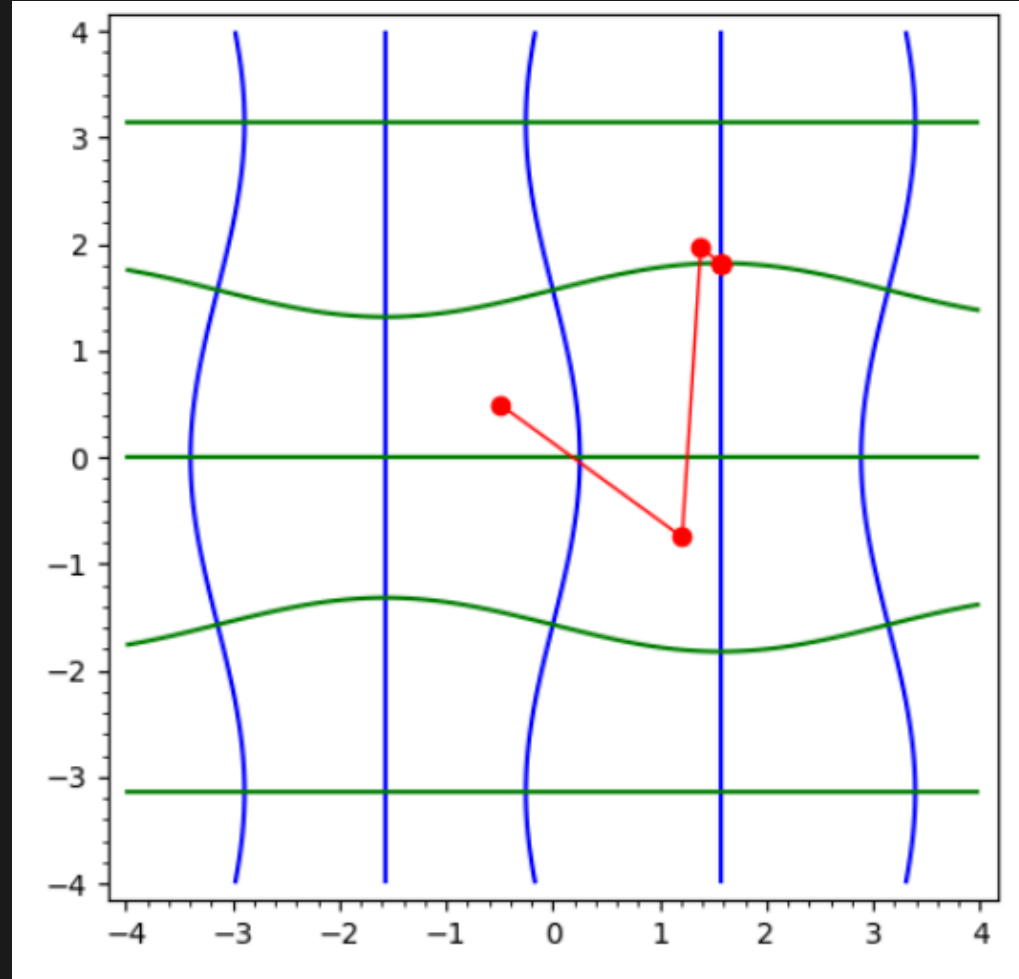
4.250000003637724

FOTOS

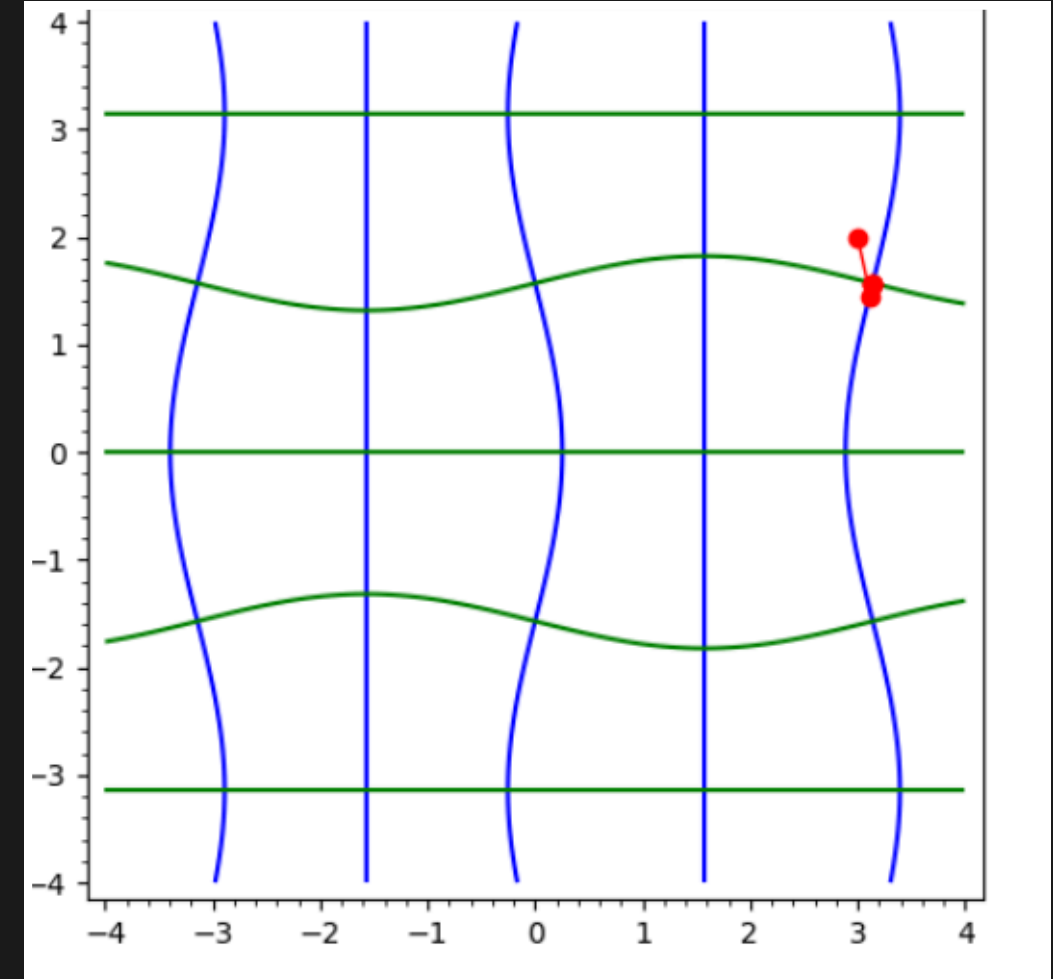
MÁXIMO



MÍNIMO



PUNTO DE SELA

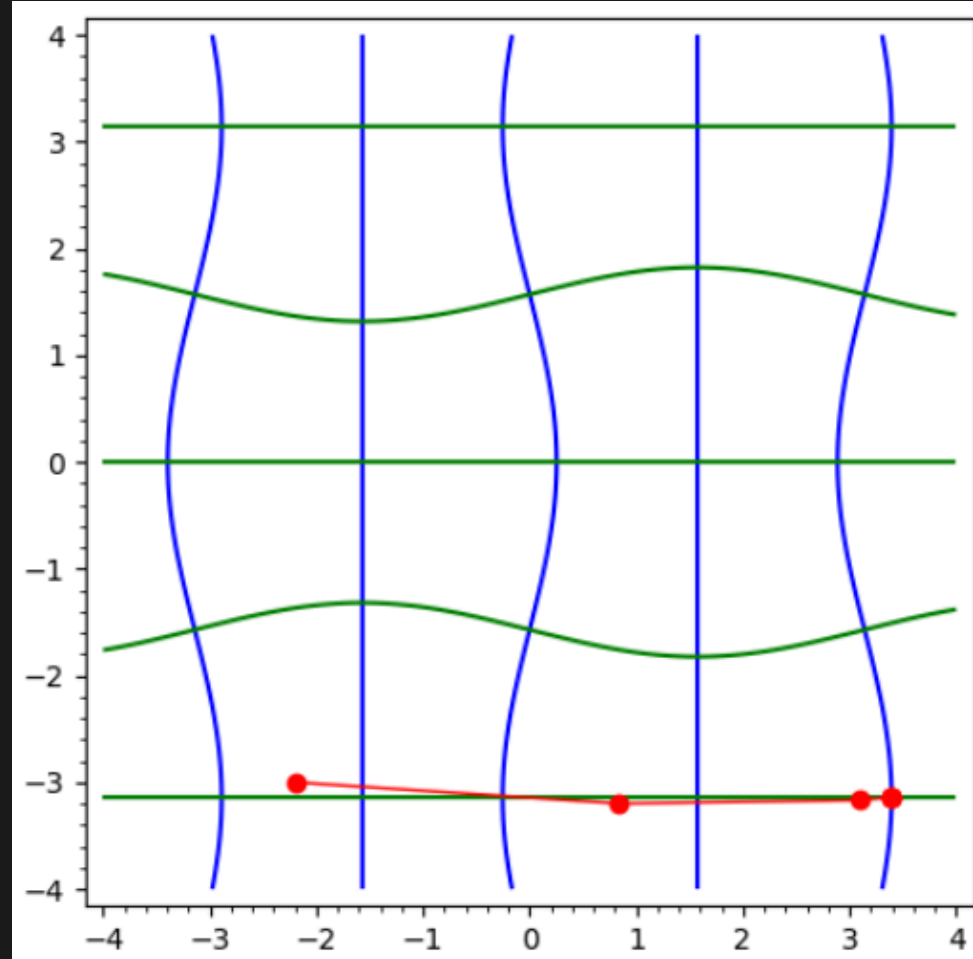


```
s0 = newton(diff(h), diff(h,2), (-2.2,-3))[0]  
s0, criterio_hessiana(h, s0)  
  
((3.394272908731872, -3.141592653589793), 'maximo')
```

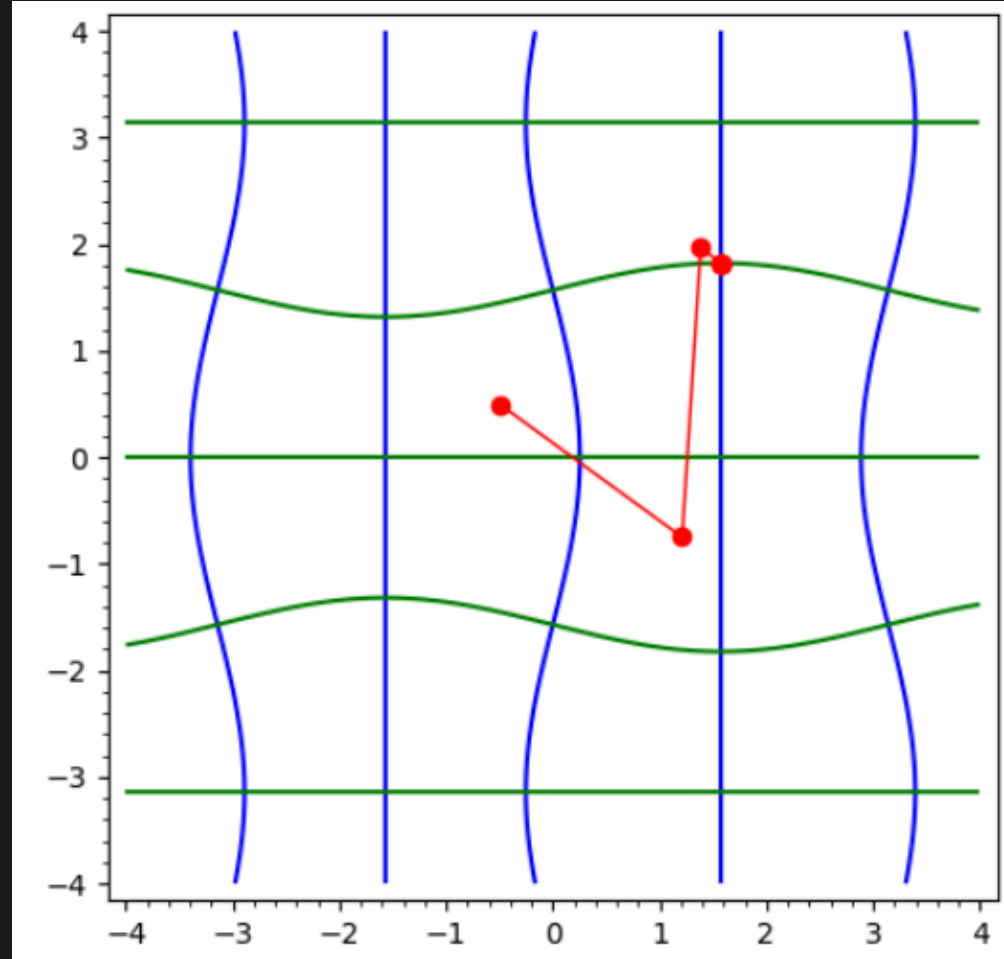
Número de iteraciones: 7

FOTOS

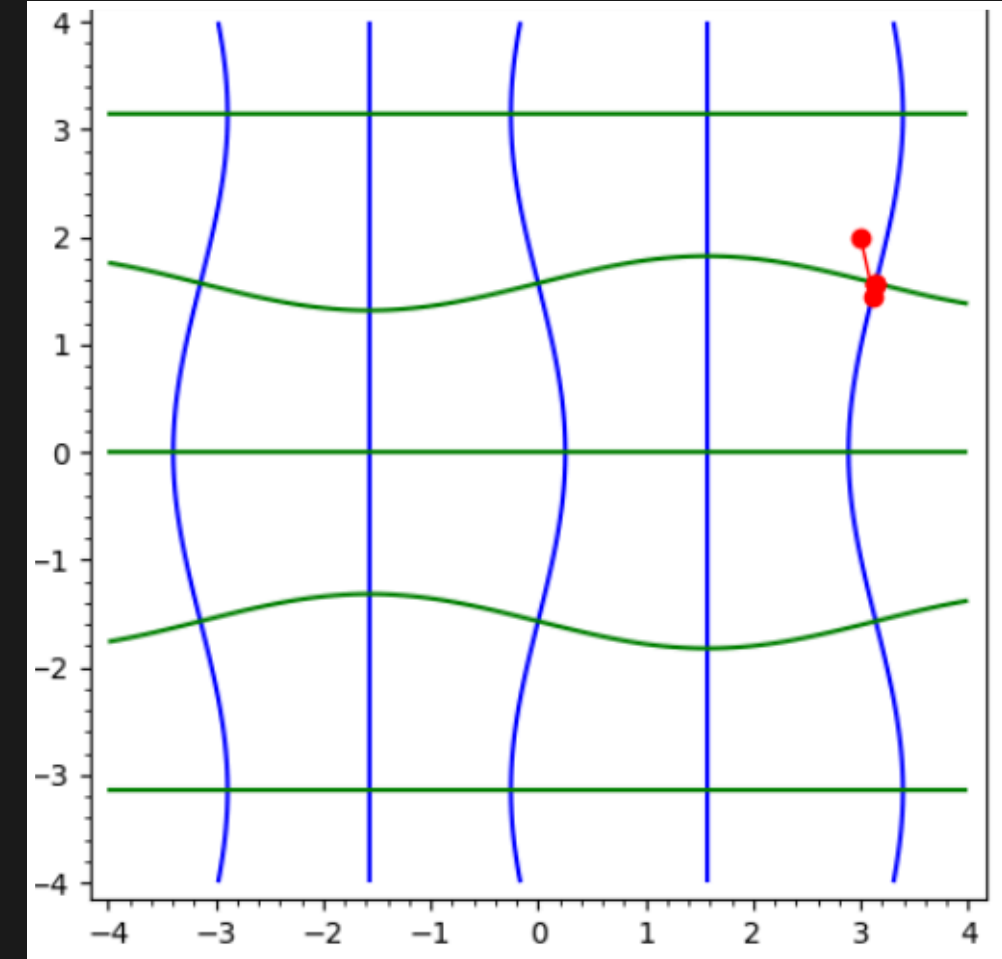
MÁXIMO



MÍNIMO



PUNTO DE SELA

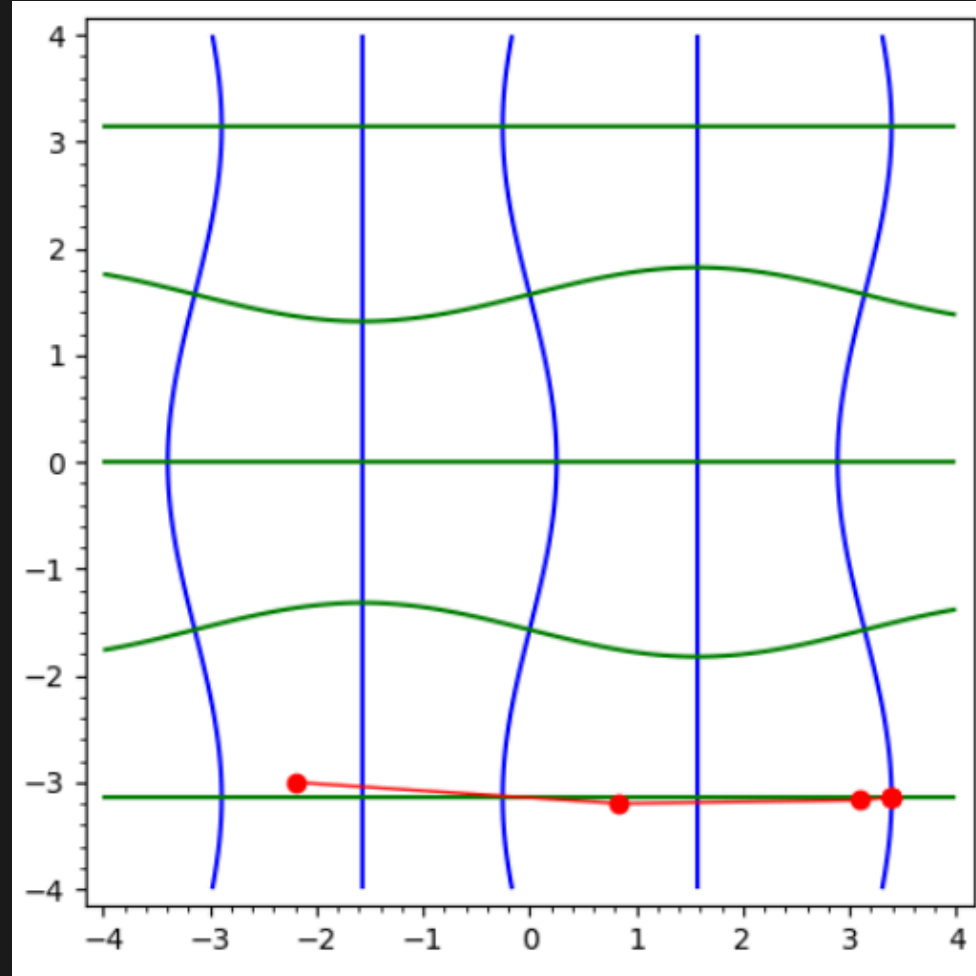


```
s0 = newton(diff(h), diff(h,2), (-0.5,0.5))[0]  
s0, criterio_hessiana(h, s0)  
  
((1.5707963267948966, 1.8234765819369754), 'minimo')
```

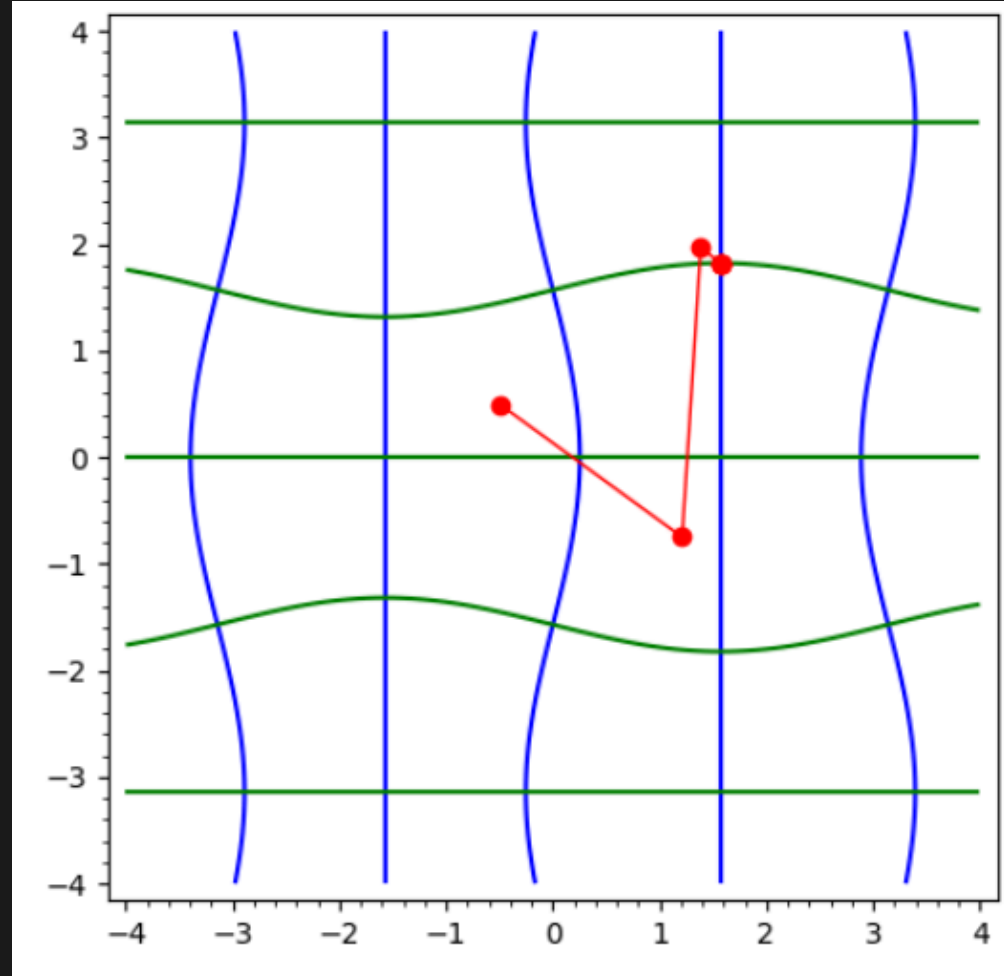
Número de iteraciones: 7

FOTOS

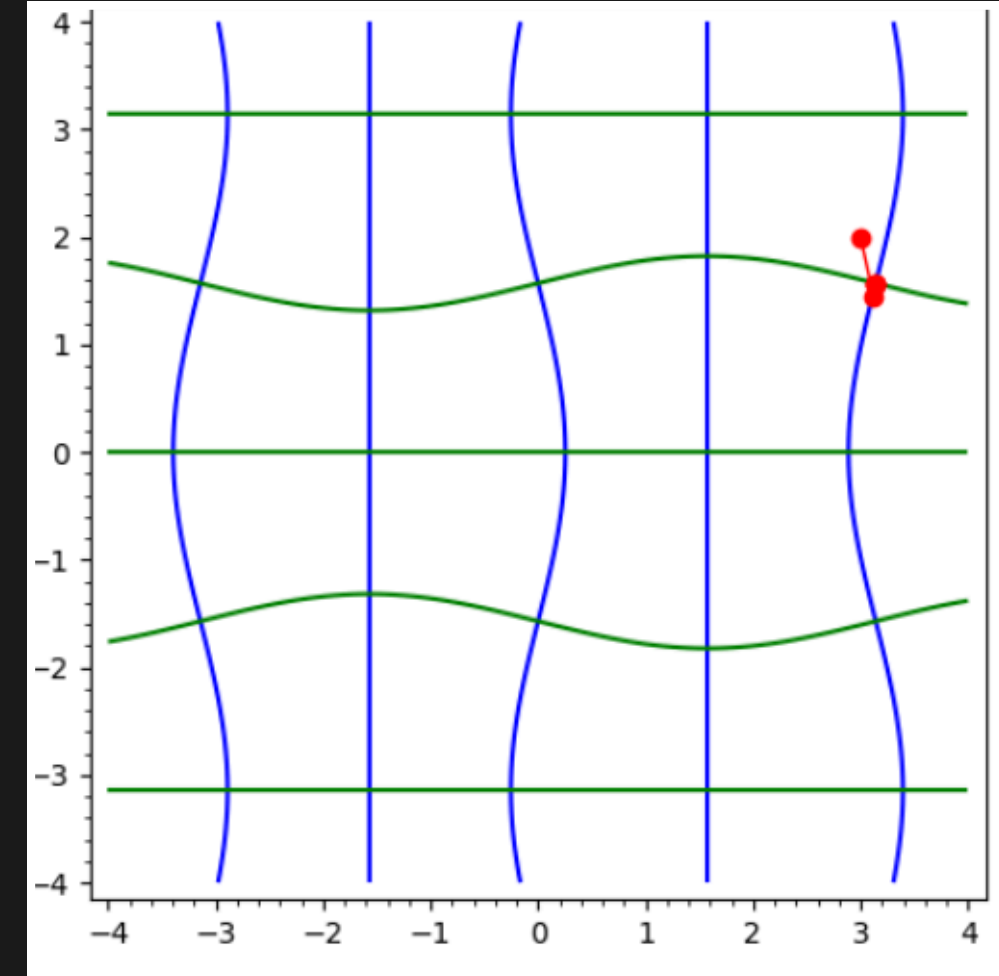
MÁXIMO



MÍNIMO



PUNTO DE SELA

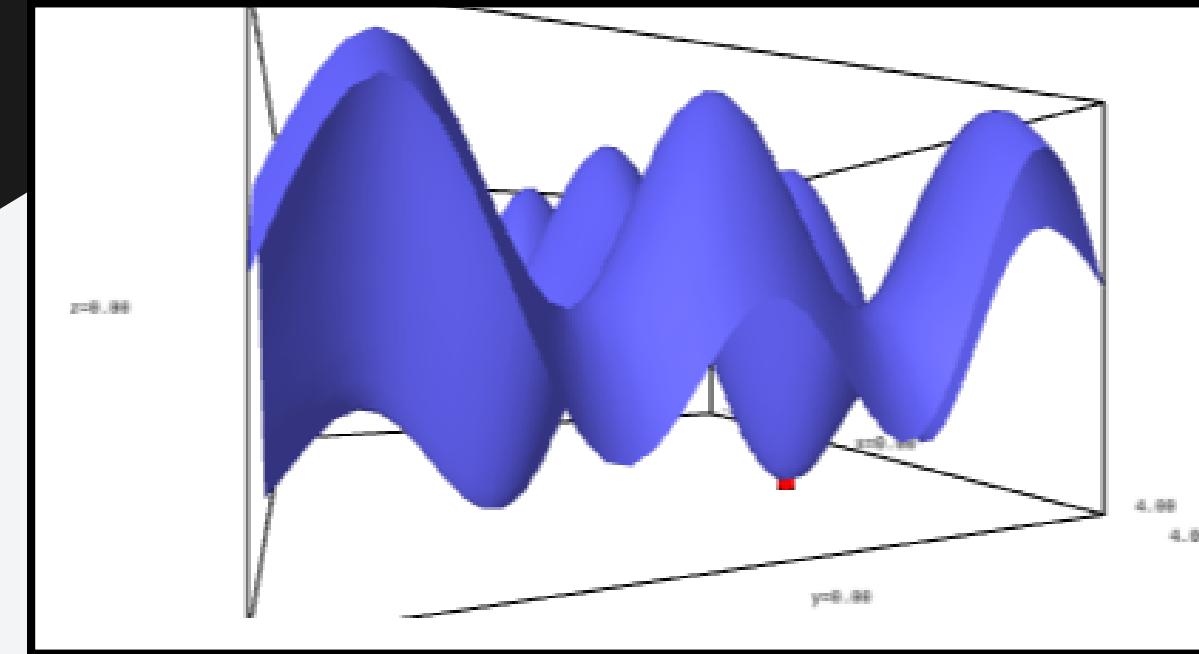


```
s0 = newton(diff(h), diff(h,2), (3,2))[0]  
s0, criterio_hessiana(h, s0)  
  
((3.141592653589793, 1.5707963267948966), 'punto de sela')
```

Número de iteraciones: 5

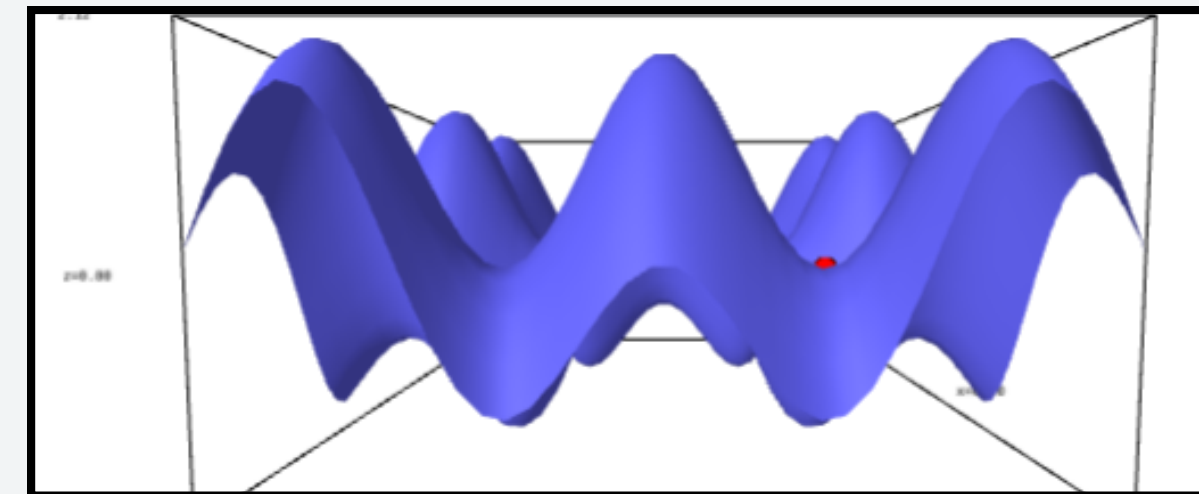
MÍNIMO

```
var('y')
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)
x=plot3d(sin(x)*cos(y)+cos(2*x) +cos(2*y),(x,-4,4),(y,-4,4))
y=point((1.5707963267948966, 1.8234765819369754,h(1.5707963267948966, 1.8234765819369754)),color='red',size=50)
x+y
```



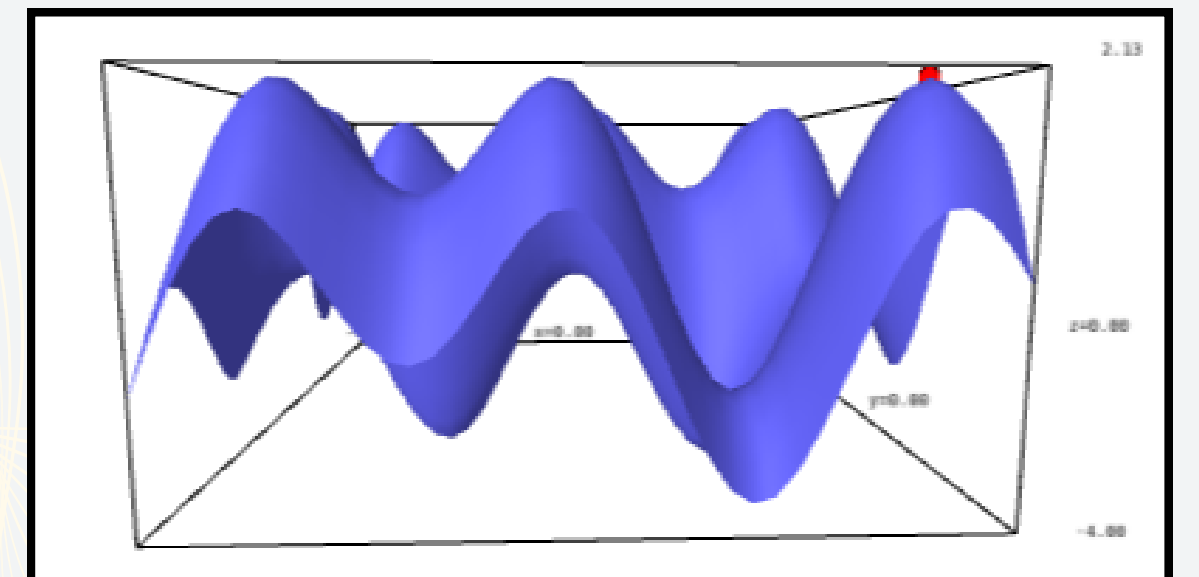
PUNTO DE SELA

```
var('y')
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)
x=plot3d(sin(x)*cos(y)+cos(2*x) +cos(2*y),(x,-4,4),(y,-4,4))
y=point((3.141592653589793, 1.5707963267948966,h(3.141592653589793, 1.5707963267948966)),color='red',size=50)
x+y
```



MÁXIMO

```
var('y')
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)
x=plot3d(sin(x)*cos(y)+cos(2*x) +cos(2*y),(x,-4,4),(y,-4,4))
y=point((3.394272908731872, -3.141592653589793,h(3.394272908731872, -3.141592653589793)),color='red',size=50)
x+y
```



DESCENSO RÁPIDO

```
def descenso(f, df, x0, xtol=1e-10, rtol=1e-12, M=100):
    """DESCENSO    Cálculo de mínimos pelo método do descenso."""
    v = [vector(RDF,x0)]
    for k in range(M):
        z = -vector(RDF,df(*v[k]))
        if norm(z) < xtol:
            return True, v[k], len(v), 'Gradiente nulo', v
        z = z/norm(z)
        #pos:0      1      2      3
        a = [None, 0.,      None, 1.]
        g = [None, f(*v[k]).n(), None, f(*(v[k]+a[3]*z).n())]
        while g[1] <= g[3]:
            a[3] = a[3]/2
            g[3] = f(*(v[k]+a[3]*z).n())
            if a[3] < xtol:
                return True, v[k], len(v), 'Sen mellora', v
            a[2] = a[3]/2
            g[2] = f(*(v[k]+a[2]*z).n())
            #pos:  0      1      2      3
            h = [None, (g[2]-g[1])/a[2], (g[3]-g[2])/(a[3]-a[2]), None]
            h[3] = (h[2]-h[1])/a[3]
            a[0] = (a[2]-h[1]/h[3])/2
            g[0] = f(*(v[k]+a[0]*z).n())
            p = g.index(min(g))
            v.append(vector(RDF,v[k]+a[p]*z))
            if abs(g[1]-g[p]) < rtol:
                return True, v[k+1], len(v), 'Iterantes proximos', v
    return False, None, len(v), 'Excedido n. maximo de iteracions', v
```

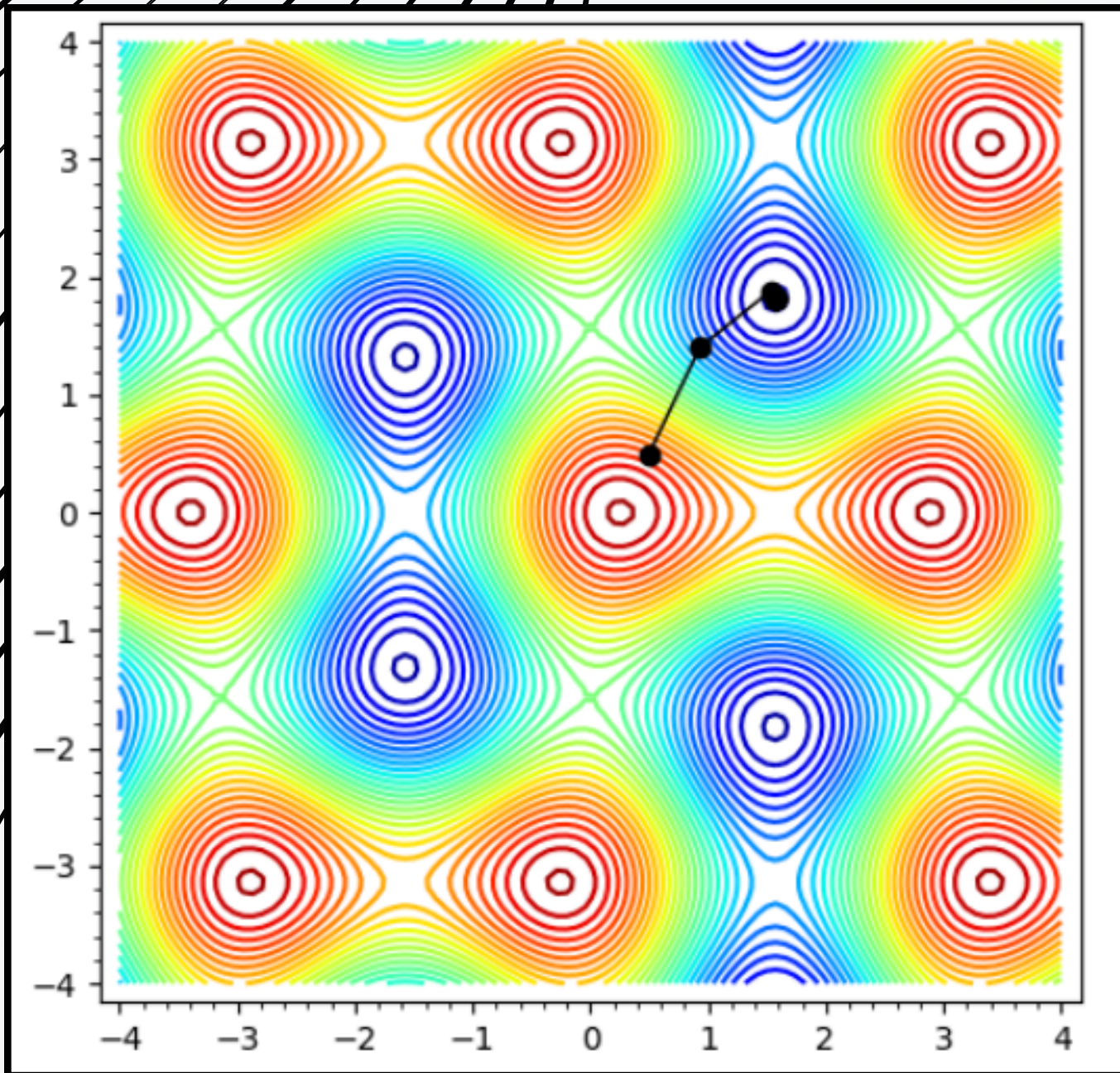
```
var('y')
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)
descenso(h, diff(h), [-0.5,0.5])[0:5]

(True,
 (-1.5707963291809788, 1.3181160678957924),
 9,
 'Iterantes proximos',
 [(-0.5, 0.5),
  (-1.3603825363741462, 1.009648791916935),
  (-1.611233067897303, 1.2729429730192765),
  (-1.567880139001508, 1.314687650069988),
  (-1.570988191282503, 1.3179059879425863),
  (-1.5707847638422057, 1.318102506402708),
  (-1.5707970760894179, 1.3181152512797545),
  (-1.5707962816755705, 1.3181160187257028),
  (-1.5707963291809788, 1.3181160678957924)])
```

```
def debuxa_nivel(f, v, args=[(x,-5,5),(y,(-5,5))],kwds={}):
    return contour_plot(f,*args, fill=False, cmap='jet', **kwds)+line(v,marker='o', color='black')

v=descenso(h, diff(h), (-0.5,0.5))[4]
debuxa_nivel(h,v,args=[(x,-4,4),(y,-4,4)],kwds={'contours':40})
```

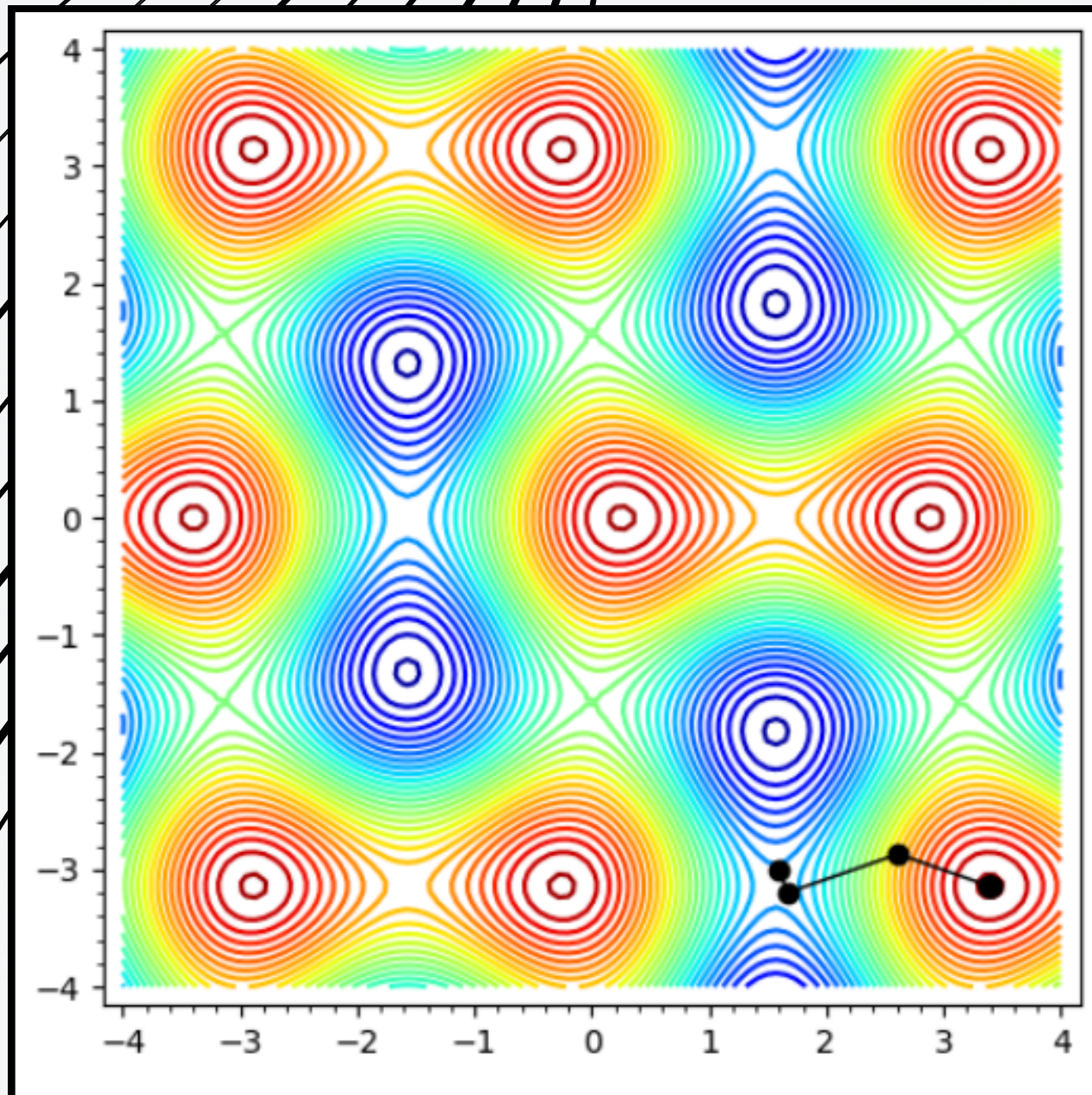

MÍNIMO



```
var('y')  
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)  
descenso(h, diff(h), [0.5,0.5])[1:3]  
  
( (1.5707963242483212, 1.8234765858745139), 9)
```

```
def debuxa_nivel(f, v, args=[(x,-5,5),(y,(-5,5))],kws={}):  
    return contour_plot(f,*args, fill=False, cmap='jet', **kws)+line(v,marker='o', color='black')  
  
v=descenso(h, diff(h), (0.5,0.5))[4]  
debuxa_nivel(h,v,args=[(x,-4,4),(y,-4,4)],kws={'contours':40})
```


MÁXIMO



```
var('y')  
h(x,y)=sin(x)*cos(y)+cos(2*x) +cos(2*y)  
descenso(-h, -diff(h), [1.6, -3])[1:3]
```

```
((3.394272909907021, -3.141592650064559), 9)
```

```
def debuxa_nivel(f, v, args=[(x,-5,5),(y,(-5,5))],kwds={}):  
    return contour_plot(f,*args, fill=False, cmap='jet', **kwds)+line(v,marker='o', color='black')
```

```
v=descenso(-h, -diff(h), (1.6,-3))[4]  
debuxa_nivel(h,v,args=[(x,-4,4),(y,-4,4)],kwds={'contours':40})
```

WEBGRAFÍA

Aula virtual de Cálculo II